OSTI-x-CARDS

Turning 235,000 DOE Scientific Papers into
Structured, Agent-Actionable Knowledge Cards

Architecture, Workflows, Templates, Parsing, Fetching, and Serving

Kukla
Hermes Agent / M1 Mac mini
`github.com/rick-stevens-ai/OSTI-x-CARDS`

October 21, 2026

Abstract

OSTI-x-CARDS is an end-to-end knowledge-extraction system that converts the U.S. Department of Energy's Office of Scientific and Technical Information (OSTI) corpus — roughly **234,705** scientific papers — into *x-Cards*: structured, machine-readable records describing the research *resources* each paper introduces or uses (datasets, trained models, software agents, instruments, user facilities, simulation codes, and computational workflows). Papers are fetched from OSTI, parsed on GPU with Marker and Nougat, read by an LLM farm that emits seven-modality cards, quality-scored against a four-component rubric, enriched by zero-hallucination cross-card fact transfer, merged into consensus *SuperCards* per real-world resource, and rendered as review/-tutorial articles. The result is served through a FastAPI web interface and a Model Context Protocol (MCP) server for direct agent access. As of this writing the system holds **311,188** x-Cards across **134,956** distinct papers and **18,862** SuperCards consolidating **155,110** member cards. This paper documents the complete system: the data model, the batch pipeline stage by stage, the LLM infrastructure, the card and review templates, the quality and enrichment methodology, the SuperCard entity-resolution strategy, and the serving layer — together with the engineering principles (zero hallucination, restartability, replace-on-improvement, honest tiering) that govern every stage.

Contents

1 Introduction and Motivation **2**
1.1 What is in an x-Card . . . 2
1.2 Current scale (live snapshot) . . . 2

2 System Architecture **4**
2.1 End-to-end data flow . . . 4
2.2 Component inventory . . . 4
2.3 Why SQLite everywhere . . . 4

3 The Data Model **6**
3.1 `cards` (osti_cards.sqlite) . . . 6
3.2 `supercard` (supercards.sqlite) . . . 6
3.3 `entities.sqlite` . . . 6
3.4 The catalog (catalog.sqlite) . . . 6

4 Stage 0–1: Catalog and Corpus Fetching **8**
4.1 Building the catalog . . . 8
4.2 Fetching full text . . . 8
4.3 The corpus-recovery loop . . . 8

5 Stage 2: Parsing PDFs to Text **9**
5.1 Why this runs on HPC . . . 9
5.2 The mmd-to-PDF auto-render workflow . . . 9

6 Stage 3: x-Card Extraction **10**
6.1 The extractor and the sync . . . 10
6.2 Multi-entity capture . . . 10
6.3 The derived-content bracketing convention . . . 10
6.4 The universal provenance block . . . 10
6.5 LLM infrastructure . . . 11

7 Stage 4: Quality Assurance **12**
7.1 The three sub-scores . . . 12
7.2 The v4 HIGH gate (current definition) . . . 12
7.3 URL validation . . . 12
7.4 The QA pipeline is non-destructive . . . 12

8 Stage 5: Enrichment **13**
8.1 The measured gap . . . 13
8.2 The source ladder (cheapest, most trustworthy first) . . . 13
8.3 Staging, not mutation . . . 13

9 Stage 6: SuperCards **15**
9.1 Feasibility (live corpus) . . . 15
9.2 The hard problem: entity resolution . . . 15
9.3 Consensus synthesis . . . 15
9.4 SuperCards as a first-class website category . . . 15

10 Stage 7: Review/Tutorial Article Synthesis **16**
10.1 The synthesizer . . . 16
10.2 Per-modality section templates . . . 16
10.3 The prompt contract . . . 16

11 Stage 8: Citation Enrichment **18**

12 Stage 9: The Serving Layer **18**
12.1 The web API (`web/app.py`) . . . 18
12.2 The MCP server (`mcp/mcp_server.py`) . . . 18
12.3 Deployment and access . . . 19

13 Engineering Principles **20**
13.1 Zero hallucination . . . 20
13.2 Restartability and chainability . . . 20
13.3 Replace-on-improvement and non-destructive writes . . . 20
13.4 Honest tiering and provenance . . . 20
13.5 Storage and durability . . . 20

14 Replication: Rebuilding the Stack from Zero **21**

15 Discussion and Future Work **21**

1 Introduction and Motivation

The DOE national-laboratory complex produces an enormous, continuously growing literature. OSTI is its public clearinghouse: journal articles, conference papers, and technical reports, each with a stable OSTI identifier and (often) a full-text PDF. For a human researcher this corpus is navigable one paper at a time. For an *autonomous agent* — the kind of system that now plans experiments, writes analysis code, and reasons over scientific infrastructure — a pile of PDFs is nearly useless. What an agent needs is not prose but *structured, grounded, queryable facts about reusable research resources*: which dataset was released, where it lives, under what license; which simulation code was used, at what version; which instrument at which facility produced the measurements; which software agent was built and what it can do.

OSTI-x-CARDS exists to manufacture exactly that layer. It reads the corpus and distills, from each paper, the *resources* it describes into a fixed schema of seven modalities. Each distilled record is an **x-Card**. Cards are then quality-graded, cross-linked, enriched, and merged so that a resource mentioned in three thousand papers (say, the VASP simulation code, or the Advanced Photon Source) becomes one authoritative **SuperCard** with an evidence-weighted consensus record and a synthesized review article. Everything is served to both humans (a browsable web site) and machines (an MCP server).

What problem is being solved

A corpus of unstructured PDFs is converted into a structured, agent-actionable knowledge base of *research resources*, with every fact traceable to a source OSTI record and nothing fabricated. The deliverable is not a search engine over papers; it is a *resource catalogue* extracted from papers.

1.1 What is in an x-Card

Each x-Card captures one research resource from one paper, in one of seven modalities. The modality determines the required fields and the review-article section template.

Modality	Captures	Example resource
`data`	Published datasets	FLUXNET2015 eddy-covariance flux data
`model`	Trained/parametric models	A crystallography refinement model
`agent`	Autonomous software agents	Peregrine (ORNL additive-mfg. agent)
`instrument`	Physical measurement devices	X-ray streak camera at APS Sector 7
`facility`	User facilities / infrastructure	Advanced Photon Source; NERSC
`simulation`	Simulation / modeling codes	VASP; LAMMPS
`workflow`	Computational pipelines	Genomics variant-calling workflow

1.2 Current scale (live snapshot)

The numbers below are read directly from the running service's `/api/stats` and `/api/supercards/stats` endpoints.

Quantity	Value
Corpus universe (Journal + Conference + Tech. Report)	234,705 papers
x-Cards extracted (real, excl. sentinels)	311,188
Distinct papers with ≥ 1 card	134,956
SuperCards (consolidated resources)	18,862
SuperCard member cards	155,110
Cards with official DOE subject classification	193,000+

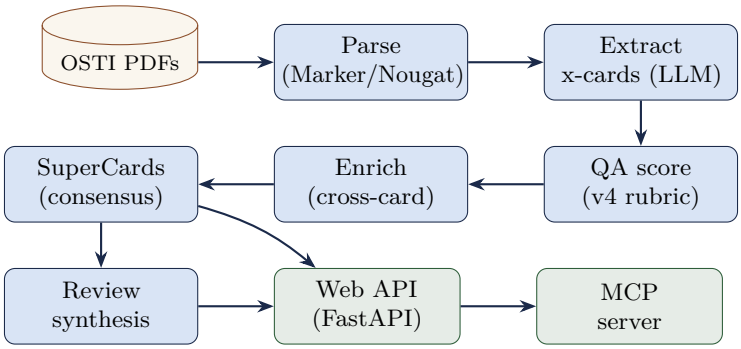
Card counts by modality: instrument 82,777; facility 71,108; simulation 66,039; workflow 50,855; model 20,036; data 18,276; agent 2,097. The corpus spans publication years 2000–2026, with the bulk (>20k cards/year) concentrated in 2018–2025.

(1) Zero hallucination. Emails, URLs, and versions are recorded only if observed in paper text, a sibling card, or a verified contact database — never pattern-constructed. **(2) Review/tutorial, not sparse outlines.** Cards and SuperCards render as synthesized explanatory articles grounded in the source papers with inline `[OSTI:id]` citations. **(3) Replace-on-improvement.** Enrichment never regresses a good card; QA writes new columns and new versions, never destructively overwrites. **(4) Restartable at every stage.** Farm jobs claim work atomically and resume; a kill loses only in-flight items.

2 System Architecture

OSTI-x-CARDS is a *batch knowledge-extraction pipeline feeding a read-serving layer*. Every stage is SQLite-backed, farm-parallel at the LLM-heavy steps, and restartable. The design deliberately separates slow, expensive, offline enrichment (the pipeline) from fast, cheap, online access (the web + MCP layer).

2.1 End-to-end data flow



The canonical flow is: **OSTI corpus (PDFs)** → Marker/Nougat parsing to `.md`/`.mmd` → LLM extraction to seven-modality cards → QA scoring → cross-card enrichment → SuperCard merging → review-article synthesis → served through the web API and MCP server.

2.2 Component inventory

The repository is organized by pipeline stage, with code only — no data — in the tree. The databases and corpus live on dedicated storage (Section 13.5).

Directory	Responsibility
`scripts/`	Catalog build + corpus fetch (Stages 0–1)
`pipeline/extraction/`	Parsed text → raw x-cards; sync to SQLite + FTS5
`pipeline/quality/`	QA scoring, URL validation, code/data-repo filters
`pipeline/enrichment/`	Cross-card fact transfer, email mining, entity resolution
`pipeline/supercards/`	Merge same-resource cards into consensus records
`pipeline/reviews/`	Synthesize review/tutorial articles
`pipeline/citations/`	OpenAlex / Semantic Scholar citation enrichment
`web/`	FastAPI browsing/search site + static frontend
`mcp/`	MCP server exposing cards to agents
`prompts/`	All LLM prompts (extraction, enrichment, review)
`docs/`	Architecture, QA rubric, strategy docs, runbooks
`datasets/`	Manifests + pointers to rebuild every data artifact
`tests/`	End-to-end + performance tests

2.3 Why SQLite everywhere

Every stage reads and writes SQLite. This is a deliberate choice: SQLite files are single-file, trivially replicated (rsync/scp between hosts), support FTS5 full-text indexing natively, and tolerate the "one long-lived writer, many readers" access pattern that the pipeline exhibits. The pipeline stages are coordinated not by a message queue but by *atomic work-claiming SQL* (e.g. "select rows where

`review_md IS NULL`") — which is what makes the whole system restartable with no external scheduler state.

3 The Data Model

Three SQLite databases carry the system's state: the **card** store, the **SuperCard** store, and the **entity** store. A fourth (the *catalog*) holds OSTI metadata and parse paths and seeds the whole pipeline.

3.1 `cards` (osti_cards.sqlite)

One row per extracted resource-card. Key columns:

- `osti_id`, `year`, `mode` (data / model / agent / instrument / facility / simulation / workflow)
- `title`, `description`, `yaml` (the full structured card body)
- `urls`, `emails`, `authors` (JSON arrays; zero-hallucination)
- `qa_version`, `qa_tier` (HIGH/MEDIUM/LOW), and the component scores `qa_completeness`, `qa_provenance`, `qa_grounding`, `qa_entity_count`, `qa_missing_fields`
- `doe_codes`, `doe_subject_names`, `doe_domains` (official DOE OSTI subject classification)
- `review_md`, `review_model`, `review_at` (the synthesized review/tutorial article and its provenance)

An FTS5 virtual table indexes title/description/body for prefix search, and is what both the website search box and the MCP `osti_search` tool query.

3.2 `supercard` (supercards.sqlite)

One row per real-world resource, merging many cards. Kept in a *sibling* DB so the per-paper card corpus stays pristine and SuperCards can be rebuilt independently.

- `super_id` (= `super-<mode>-<slug>`, stable/idempotent), `canonical_name`, `mode`, `member_count`, `first_year/last_year`
- `consensus_json` — evidence-weighted field consensus with support/dissent counts
- `urls`, `emails`, `labs` (observed across members)
- `evidence` — pre-joined member-paper descriptions, for review synthesis
- `review_md`, `review_model`, `review_at`
- join table `supercard_member(super_id, osti_id)`

3.3 `entities.sqlite`

Entity and alias resolution (names/tools/datasets) that feeds SuperCard merging. This is the layer that decides "Advanced Photon Source" = "APS" = "APS at Argonne" are the same resource (Section 9).

3.4 The catalog (catalog.sqlite)

The catalog's `papers` table is the universe and the index: `osti_id`, `title`, `product_type`, `year`, `primary_lab`, `subjects_json`, and — critically — `md_path` and `mmd_path`, the on-disk locations of the Marker and Nougat parses. The authoritative corpus total is **234,705** papers, defined as `product_type` ∈ {Journal Article, Conference, Technical Report} — books, theses, and the large Materials-Project dataset records are deliberately excluded.

A standing hazard: the catalog path drift

After any re-shard of the parsed `.md`/`.mmd` files on disk, the catalog's `md_path`/`mmd_path` columns go stale (they point at the pre-move location). This is the single most common drift bug in the corpus. A dedicated repair toolkit reconciles disk-vs-catalog, rewrites paths to disk truth idempotently (backing up the catalog first), and re-shards stray files into their `<year>/` directories. Any stage that writes parse paths must batch its writes and defer rather than fight the long-lived recovery writer that holds the WAL lock.

4 Stage 0–1: Catalog and Corpus Fetching

4.1 Building the catalog

`scripts/build_catalog.py` bulk-pulls OSTI metadata into the catalog from the public OSTI JSON API (`https://www.osti.gov/api/v1/records`), parameterized by a set of DOE labs and a year range. It iterates `(lab, year)`, paginates `/records` at `rows=500` ordered by publication date, normalizes the organization list into a `primary_lab`, and UPSERTs each record into the `papers` table. `scripts/init_catalog.py` creates the schema first.

```
python scripts/init_catalog.py
python scripts/build_catalog.py --year-start 2000 --year-end 2026 --labs ANL,ORNL
# Catalog path resolves from --catalog | $OSTI_CATALOG | data/catalog.sqlite
# OSTI etiquette: a contact email goes in the User-Agent via $OSTI_CONTACT_EMAIL
```

4.2 Fetching full text

`scripts/fetch_osti_corpus.py` retrieves PDFs via the OSTI PURL endpoint (`https://www.osti.gov/servlets/purl/`) plus a discovery pass, and writes them year-sharded as `<year>/<osti_id>.pdf`.

OSTI fetches must run from a host OSTI does not blackhole. A home consumer IP is rate-limited into silent zero-byte responses; the production fetch therefore runs from a DOE-network host (e.g. a CELS DGX). The PURL endpoint is itself flaky — off-site redirects can silently yield a 0-byte file — so every fetch validates the `%PDF` magic header and a minimum size floor before accepting a download. The authoritative "do we already have this paper?" test is a SHA-256 of the file bytes against the manifest, *never* the basename (basenames collide across provenance caches).

4.3 The corpus-recovery loop

The gap between "papers known to exist in OSTI" and "PDFs physically on disk" is closed by an iterative recon → fetch → diagnose → recover loop. Each missing paper is placed in a WHY-bucket (PURL 404, 403, non-PDF landing page, publisher redirect, no DOI, . . .) and each bucket has a priority-ordered recovery strategy (Unpaywall, OpenAlex, Semantic Scholar, direct-publisher, browser-context CDP fetch for bot-walled publishers, institutional-IP exit node for licensed content). A state DB owns every `(osti_id, strategy, outcome)` triple and runs strategies in order until exhaustion. Genuinely closed-access or withdrawn records are recorded as such, not retried forever.

5 Stage 2: Parsing PDFs to Text

An LLM cannot read a PDF's layout; it reads text. Stage 2 converts each PDF into two complementary plain-text renderings:

- **Marker** → `.md` — a fast, layout-aware Markdown rendering that handles headings, lists, and most tables well.
- **Nougat** → `.mmd` — a transformer OCR model (Meta's Nougat) that excels at mathematical notation and dense scientific typesetting, producing "mathpix markdown."

The goal is *one `.md` and one `.mmd` per paper*. Having both gives the downstream extractor a fallback: where Marker drops a table, Nougat often keeps it, and vice-versa. Parse paths are recorded in the catalog's `md_path` and `mmd_path` columns.

5.1 Why this runs on HPC

Parsing is the only GPU-bound stage. Marker and Nougat run on A100 (ALCF Polaris, CELS DGX, or a local 8×A100 host) and Intel PVC (Aurora) in restartable batch jobs. The jobs follow the restartability discipline (Section 13): atomic per-item work-queue claims, an external cron-driven reaper that recovers orphaned "running" rows, and self-chaining resubmission to keep N jobs in flight until the pending count reaches zero. A walltime kill therefore loses only the handful of papers in flight at that instant.

GPU co-tenancy and the OOM cascade

On a shared multi-GPU host, resident inference servers frequently occupy 27–55 GB on random GPUs. A parse worker that lands on a memory-tight GPU will OOM, and once its CUDA context is destroyed it can mark every remaining claimed item FAILED — leaving orphan locks that make surviving workers exit early. The pre-flight check queries per-GPU free memory and pins one worker per GPU with >60 GB free via `CUDA_VISIBLE_DEVICES`. A 0-byte `.mmd` from the small Nougat model is a genuine parse bailout (repetition collapse or an image-only PDF), not a pipeline bug; those papers are flagged for a base-model redo rather than retried in place.

5.2 The mmd-to-PDF auto-render workflow

For the website's "view the source paper" feature, parsed `.mmd` files are rendered back to PDF. An operator on the serving host triggers idempotent remote rendering over the mesh, then incrementally pulls the rendered PDFs into the site's tree. This keeps the render farm's output flowing to the browse layer without a monolithic batch step.

6 Stage 3: x-Card Extraction

This is the heart of the system: reading parsed text and emitting structured cards.

6.1 The extractor and the sync

The production extractor (the "extract-o-matic" family) reads a paper's parsed text and, for each of the seven modalities present, emits a structured card. `pipeline/extraction/sync_cards.py` then syncs the farm-produced cards back to `m1` and (re)builds the SQLite FTS5 index. It is idempotent — safe to run on a timer while extraction is still producing cards — and it skips `NO_X_FOUND` sentinels so that only real cards land in the DB.

`sync_cards.py` **DROPS and RECREATES** the `cards` table on a full sync. Enrichment, QA, and review columns are therefore applied *after* a full sync completes — otherwise they are silently lost. The system's answer to this is the sibling-DB pattern (SuperCards, enrichment staging, citations all live in their own files) plus a strict stage ordering in the replication runbook.

6.2 Multi-entity capture

A paper that describes three instruments must yield *three* instrument entries, not just the first. The extraction prompt emits a *list* of entities per modality, superseding the legacy single-block, first-match-only behavior. Each entity is independently completeness-scored, and the card records its `entity_count`. Within a card, the same named entity mentioned repeatedly is merged to one entity.

6.3 The derived-content bracketing convention

A global rule distinguishes paper-grounded fact from model inference:

- **Direct** content (copied/closely paraphrased from the paper) is written plain.
- **Derived** content (inferred, normalized, or synthesized) is wrapped in brackets with its source, e.g.
  `license: "[derived: CC-BY-4.0 from journal default]"`.
- **Unknown** values are written `"UNKNOWN"` — never guessed unbracketed.

A reader, and the quality scorer, can thus always tell what the paper said from what the model inferred.

6.4 The universal provenance block

Every card carries a provenance block tying it back to the source:

```
source:
  osti_id: "<id>"                        # REQUIRED, direct
  title: "<paper title>"                 # REQUIRED, direct
  doi: "<doi or UNKNOWN>"
  url: "<landing/pdf url or UNKNOWN>"
  authors: ["Last, First", ...]          # REQUIRED, direct
  author_emails: ["a@lab.gov", ...]      # direct if present
  affiliations: ["Argonne National Laboratory", ...]
```

```
primary_lab: "<DOE lab or UNKNOWN>"  # derived(bracket) allowed
funding:
  - agency: "DOE Office of Science"
    award: "DE-AC02-06CH11357"
funding_text: "<verbatim acknowledgement>"
```

6.5 LLM infrastructure

Extraction and review synthesis run on the **pecans qwen3.8-27b farm** — an endpoint pool of roughly 17 live OpenAI-compatible `/v1/chat/completions` servers, authenticated with a bearer key. Jobs are sharded with `-shard M/N` (by `rowid % N`) across workers and claim work atomically, so scaling is a matter of launching more shards. A high-quality fallback path routes to Argo Sonnet through a LiteLLM aggregator on `localhost:4000`. For review synthesis, `enable_thinking=False` — the tokens should become article text, not reasoning trace.

7 Stage 4: Quality Assurance

Every card is scored so consumers can distinguish a complete, well-grounded card from a thin one. The scorer (`pipeline/quality/qa_score.py`) computes the deterministic/structural sub-scores; the LLM-judged grounding sub-score is computed separately (regex is never used for judgment).

7.1 The three sub-scores

- **completeness** = filled required fields / total required fields, averaged across entities. A field is "filled" if non-empty and not a bare `UNKNOWN` (bracketed-derived counts as filled).
- **provenance** = weighted presence of the provenance block (osti_id .1, title .1, authors .2, emails .15, affiliations .15, funding .3).
- **grounding** = fraction of entity-defining claims traceable to the parsed full text — LLM-judged, not regex.

7.2 The v4 HIGH gate (current definition)

The quality definition evolved through several of Rick's corrections; the current authoritative gate is a four-component test:

> A card is **HIGH** when it has a validated/actionable **URL**, author **names**, at least one author **email**, and key **technical information**. Funding acknowledgement is explicitly *not* a quality trigger ("nice to have"). MEDIUM and LOW fall out below the respective thresholds, and any hallucinated or ungrounded entity forces LOW. The tier is written honestly — a LOW card is labeled LOW; the distinction is surfaced, not hidden.

The per-modality required-field tables (what counts toward completeness for a `data` card vs. an `instrument` card vs. a `simulation` card) are specified in `docs/QA_RUBRIC.md`. For example, a `data` card requires dataset name/type, short and full description, provider, license, access, repository URL, size/format, and version; a `simulation` card requires code name/type, role in paper, descriptions, developer, version, license, code URL, and physics/inputs.

7.3 URL validation

`pipeline/quality/url_validate.py` classifies and HTTP-verifies the URLs on each card, setting a `url_validated` flag that feeds the HIGH gate. Companion filters (`code_repo_filter.py`, `data_repo_filter.py`) identify cards that carry an actionable code or data repository, powering the website's "Has code repository" / "Has data repository" facets.

7.4 The QA pipeline is non-destructive

QA writes *new* `qa_*` columns and new re-extracted card versions; it never overwrites the existing corpus. Every stage is idempotent and restartable, claims work atomically, and writes a ledger.

8 Stage 5: Enrichment

Extraction leaves gaps: an author's email the extractor missed, a license present in a sibling paper but not this one, a tool's developer stated in one card and absent in another. Enrichment fills those gaps *without inventing anything.*

8.1 The measured gap

Against a live snapshot of 232,035 cards / 103,017 papers:

Component	Coverage	Status
Names (authors)	95.3%	nearly solved
Actionable URL (pre-validation)	71.0%	validate + recover
Emails	52.1% of papers	49,358 papers lack any email
Key technical info (comp. ≥ 0.85)	20.9%	dominant limiter

The most-missing technical fields corpus-wide are location (116k), license (110k), version (69k), access (53k), and workflow steps (40k).

8.2 The source ladder (cheapest, most trustworthy first)

Email sources.

1. **S-E1 Parse-mining** — 61% of email-less papers have an author email printed in their *own* parse that the extractor missed. An author-email classifier mines the header/affiliation region and rejects boilerplate (`reports@*.osti.gov`, `webmaster@`, generic ordering addresses). Biggest win, zero cost, zero hallucination.
2. **S-E3 Cross-card transfer** — the same author appearing in another paper that *does* have their email; the observed address is transferred, person-name keyed.
3. **S-E4 Web** (OpenAlex / Crossref / ORCID) — resolve the corresponding author to a verified email from an authoritative record; cite the source URL. Last resort.
4. **Never** pattern-construct `first.last@lab.gov`. Verified-or-empty.

Technical-info sources. Re-parse-mining of fields that are in the parse but were dropped; cross-card consensus transfer (14,115 named entities appear in more than one card, covering 48% of the corpus); cited web enrichment with live-validated URLs; and tightly-guarded parametric facts about well-known entities ("LAMMPS is C++", "APS is at Argonne") — never for URLs, emails, versions, or access.

8.3 Staging, not mutation

`enrich_crosscard.py` transfers a consensus field value to member cards that lack it, writing to a *staging* table `card_enrichment(osti_id, mode, field, value, source, support, created_at)` — it never mutates the `cards` table. A value is transferred only when consensus support meets a minimum and the target genuinely lacks the field; the operation is idempotent. `email_mine.py` emits JSONL of mined author emails, classifier-scored, for merge. Every enriched value carries a bracketed provenance tag (`[enriched:cross-card]`, `[enriched:web <url>]`) so the original paper content stays visibly untouched.

The regex hard rule

Enrichment and QA obey a standing rule: *no regex when an LLM will do better.* Regex is permitted only for genuinely mechanical fixed-schema parsing (splitting YAML keys, extracting a URL host). Any field that feeds a downstream judgment, ranking, or selection — grounding, entity detection, field derivation, email classification — uses the LLM. The cascade pattern keeps cost down: a cheap model triages binary flags, a strong model does deep extraction only on the hits.

9 Stage 6: SuperCards

A resource like VASP, LAMMPS, or the Advanced Photon Source is described hundreds to thousands of times across the corpus. Each mention is partial. A **SuperCard** is the consolidated, canonical record of *one* real-world resource, synthesized from *every* card that mentions it.

9.1 Feasibility (live corpus)

Of 140,364 distinct named entities, **14,115 appear in more than one card** — these are SuperCard-eligible and cover 111,222 cards (48% of the corpus). The heavy hitters are exactly the high-value shared infrastructure: facilities (APS, ALS, NERSC, SNS, OLCF) and codes (VASP, LAMMPS). The long tail of single-mention entities becomes *singleton* SuperCards — still useful as the canonical entry, but without cross-card consensus.

9.2 The hard problem: entity resolution

Name-exact matching is the floor but insufficient; aliases, acronyms, and versions must be reconciled ("Advanced Photon Source" = "APS"; "VASP" = "Vienna Ab initio Simulation Package" = "vasp 5.4"). The resolution pipeline is *blocking* → *matching* → *clustering*:

1. **Normalize** the name (lowercase, strip version/punctuation) into a cheap blocking key.
2. **Alias table** — curated plus auto-detected acronym↔ expansion (from cards that contain both forms, e.g. "Advanced Photon Source (APS)").
3. **LLM adjudication within a block** (not regex) — for candidate clusters sharing a key or alias, an LLM confirms "same resource? yes/no/variant" from the cards' type, developer, and description. This is the judgment step; it runs on the farm.
4. **Version handling** — same base resource, different version → one SuperCard with a `versions:` list (VASP 5.4 and 6.3 are one VASP SuperCard).
5. **Cluster → SuperCard id** = `super-<mode>-<slug>`, stable and idempotent.

9.3 Consensus synthesis

For each cluster the canonical record is built by *evidence-weighted consensus*, not concatenation. Scalar fields (developer, license, language, location) take the value with the most support across members, recording agreement count and dissenting values. URLs and emails are the validated union, with a canonical one promoted. Descriptions are LLM-synthesized from the richest members. The provenance block becomes the *usage graph*: member cards, member count, first/last seen, labs, and authors. Conflicts are surfaced (`license_candidates: [...]`), never silently averaged away. A SuperCard asserts only facts present in at least one member card or verified via live web — the same anti-hallucination law as enrichment.

9.4 SuperCards as a first-class website category

SuperCards are a separate, first-class browse category on the site, distinct from per-paper cards, with their own search, filter, and detail pages. A SuperCard detail page shows the consensus record plus the list of member papers (the usage graph), and the MCP server exposes `supercard_search`/`supercard_get`. SuperCards also feed the knowledge graph as canonical nodes with member cards as edges.

10 Stage 7: Review/Tutorial Article Synthesis

A card's structured fields are for machines. For humans (and for grounding an agent's reasoning), each card and SuperCard also renders as a **review/tutorial article** — flowing explanatory prose, not a sparse outline of fields.

10.1 The synthesizer

`pipeline/reviews/review_synth.py` runs on the qwen3.8-27b farm. It supports two kinds:

```
# SuperCards first (best evidence):
python pipeline/reviews/review_synth.py --kind super --workers 34
# then individual cards, sharded across workers:
python pipeline/reviews/review_synth.py --kind card --shard 0/4 --workers 20   # x4
```

It is restartable (atomic claim via `review_md IS NULL`), zero-hallucination, and runs with `enable_thinking=False`. SuperCard articles are synthesized first because they have the richest evidence; individual-card articles follow, sharded by `rowid % N`.

10.2 Per-modality section templates

The article structure differs by modality — a dataset review is not a facility review. The templates are defined in code and mirrored in `prompts/`. For example:

Modality	Section template
`data`	Overview · What the Dataset Contains · How It Is Used in Research · Access & Format · How to Work With It (tutorial) · Provenance & Citation
`simulation`	Overview · Method & Physics · How It Is Used in This Corpus · Capabilities & Numerics · Typical Workflow (tutorial) · Access & Licensing
`facility`	Overview · Capabilities & Instruments · Science Enabled · How Researchers Use It · How to Get Access (tutorial) · Operation & Stewardship
`model`	Overview · Architecture & Method · What It Predicts/Does · How It Is Used · How to Use It (tutorial) · Access & Licensing · Representative Works

10.3 The prompt contract

The individual-card prompt builder (verbatim from the pipeline) feeds the paper description and the extracted YAML as source facts, instructs the model not to contradict them but to expand into prose, and enforces the rules:

```
Write a substantive review/tutorial article in Markdown with these sections:
{per-modality sections}

Rules:
- Flowing explanatory PROSE like a tutorial - NOT a bullet outline, NOT sparse.
- Explain what the resource is, how it works, how a researcher would use it.
- Do NOT invent URLs, versions, emails, or facts absent from the description/card.
```

```
- 350-600 words (cards) / 600-900 words (SuperCards). Start with: "# {title}".
```

Articles carry inline `[OSTI:id]` citations so every synthesized claim points back to the paper(s) it came from, and the website renders those as citation chips that link to the source card.

11 Stage 8: Citation Enrichment

To let users rank and weight resources by scholarly impact, each paper's citation count is fetched from OpenAlex (with Semantic Scholar as a secondary source), keyed on DOI, into `citations.sqlite`.

Citation counts come exclusively from OpenAlex / Semantic Scholar keyed on the DOI — an LLM is *never* asked how many times a paper was cited. OpenAlex enforces a per-IP daily budget; when the serving host is capped the fetch runs from a fresh-quota institutional IP. Likewise, science-domain classification comes from the authoritative DOE OSTI subject codes (`subjects_json` in the catalog), mapped to domains such as Biology, HEP, Materials, Fusion, and Computer Science — not from LLM free-text classification.

12 Stage 9: The Serving Layer

The read side is two components over the same SQLite stores: a human-facing web app and a machine-facing MCP server.

12.1 The web API (`web/app.py`)

A FastAPI + uvicorn service on port 8094, bound `0.0.0.0`, backed by the card DB's FTS5 index and the SuperCard DB. Its route surface:

Route	Purpose
`/api/stats`	Corpus totals + breakdown by mode and year
`/api/search`	FTS5 search with mode/year/repo facets
`/api/card/{year}/{mode}/{id}`	Full card (all fields + YAML)
`/api/download/...`	Download one card's YAML
`/api/bulk`	Bulk export of a filtered set (zip)
`/api/sources/{year}/{id}`	Which source artifacts exist (PDF/.md/.mmd)
`/api/pdf, /api/raw, /api/render, /api/renderpdf`	Serve original PDF, raw parse, or rendered view
`/api/supercards/stats`	SuperCard totals + by-mode breakdown
`/api/supercards/search`	Search SuperCards (min-member filter)
`/api/supercard/{super_id}`	SuperCard detail + member papers
`/api/access/stats`	Access-log analytics

A middleware logs every request durably to `access_log.sqlite` (recovering the real client IP), surfaced via `/api/access/stats`. The static frontend (`index.html`, `supercards.html`, `examples.html`) renders review articles as the primary view, with the structured YAML available on demand.

12.2 The MCP server (`mcp/mcp_server.py`)

The MCP server wraps the *same* FTS5 index and source-artifact store that powers the website, so agent tools behave identically to the HTTP API. It uses the MCP 2.x SDK over streamable-HTTP (or stdio for local dev) and exposes:

- `osti_stats` — corpus-wide statistics and breakdowns
- `osti_search` — FTS5 prefix search over title/description, with mode/year filters

- `osti_get_card` — one full card (all fields incl. complete YAML)
- `osti_get_paper_cards` — every card for a single paper
- `osti_paper_text` — the parsed full text of the baseline paper (`md` or `mmd`)
- `osti_list_sources` — which artifacts exist for a paper
- `osti_bulk_export` — base64 bulk export of a filtered set

12.3 Deployment and access

The web service runs under a `launchd` job with `KeepAlive`; it is recycled with a `launchctl kickstart`. Public access is via a Tailscale Funnel hostname (no VPN required for external users); the tailnet address serves internal clients. Private network details are never committed to the public repository — endpoint references use the Funnel hostname only.

13 Engineering Principles

Four disciplines recur at every stage and are worth stating on their own.

13.1 Zero hallucination

The system's credibility rests on this. Emails, URLs, and versions are recorded only when observed in the paper text, a sibling card, or a verified contact DB. The one previously-used contact database is now exhausted, so the live sources are parse-mining, cross-card transfer, and cited web resolution. Pattern-constructed addresses (`first.last@lab.gov`) are forbidden outright — the policy is verified-or-empty. Derived values are always bracketed with their source so a reader can separate fact from inference.

13.2 Restartability and chainability

Every long-running stage is designed for a walltime kill: atomic per-item work claims, an external reaper that recovers orphaned "running" rows, and self-chaining resubmission that keeps N jobs in flight until pending reaches zero. No stage relies on a job epilogue for chaining (a walltime SIGKILL never runs the epilogue). This is why the pipeline can run unattended across HPC queues with hard walltime caps.

13.3 Replace-on-improvement and non-destructive writes

Enrichment never regresses a good card. QA and enrichment write *new* columns and *staging* tables rather than overwriting the card corpus; superseded cards remain, versioned. The one destructive operation (the `sync_cards.py` table rebuild) is fenced by strict stage ordering and the sibling-DB pattern so that derived work is never collaterally destroyed.

13.4 Honest tiering and provenance

A LOW card is labeled LOW. Quality tiers are surfaced, not hidden, so a consumer can choose a precision/recall point. Every fact traces to a source OSTI record, every enriched value to its enrichment source, every review claim to the paper it came from. Three-tier corpus replication (a primary external drive, a DGX shared-storage mirror, and purgeable HPC scratch) protects the bytes; the derived DBs are rebuildable from the corpus at any time.

13.5 Storage and durability

The source PDFs are stored year-sharded (`<year>/<osti_id>.pdf`) with the primary copy on a dedicated external drive, a mirror on DGX shared storage, and scratch staging on HPC. The catalog (∼1.5 GB) and the parsed `.md`/`.mmd` co-locate with the PDFs. The derived databases — cards (∼2.7 GB), SuperCards (∼192 MB), entities (∼144 MB), citations, enrichment staging — are the pipeline's output and can be regenerated from the corpus. No data lives in the Git repository; `datasets/` holds only manifests and pointers.

14 Replication: Rebuilding the Stack from Zero

The complete from-scratch sequence, each stage restartable:

Stage	Command	Produces
0	`scripts/build_catalog.py`	catalog.sqlite
1	`scripts/fetch_osti_corpus.py`	`<year>/<id>.pdf`
2	Marker + Nougat (GPU batch)	`.md` / `.mmd`
3	`extraction/sync_cards.py`	osti_cards.sqlite + FTS5
4	`quality/qa_score.py`, `url_validate.py`	qa_* columns
5	`enrichment/entity_resolve.py`, `enrich_crosscard.py`, `email_mine.py`	entities + staging
6	`supercards/supercards_build.py`	supercards.sqlite
7	`reviews/review_synth.py`	review_md
8	`citations/citation_fetch.py`	citations.sqlite
9	`uvicorn app:app`; `mcp_server.py`	live site + MCP

Prerequisites: a Python 3.13 venv (`pip install -r web/requirements.txt`), access to the qwen3.8-27b farm or the Argo aggregator, a few hundred GB for the corpus plus ∼3.5 GB for derived DBs, and A100/PVC GPU access for parsing. Verification runs `tests/test_site_e2e.py` and spot-checks `/api/stats` and `/api/supercards/stats`.

Critical ordering constraint

Because `sync_cards.py` drops and recreates the `cards` table, Stages 4–8 must run *after* a full extraction sync completes — or the enrichment, QA, and review work will be lost. The sibling-DB layout (SuperCards, enrichment, citations in their own files) is what makes this safe in practice.

15 Discussion and Future Work

OSTI-x-CARDS demonstrates that a large, heterogeneous scientific corpus can be turned into a structured, agent-actionable resource catalogue under a strict zero-hallucination regime, at a scale (hundreds of thousands of cards) that is only feasible with a parallel LLM farm and a restartable, SQLite-coordinated pipeline. The dominant remaining limiter is *key technical information* completeness (∼21% of cards at the HIGH bar), which the enrichment thread attacks through re-parse-mining and cross-card consensus transfer. Each percentage point of HIGH% is required to be traceable to a named enrichment module and a verifiable source — no point earned by relabeling or fabricated fields.

Active and planned directions include: completing the author-email parse-mining pass (which recovers ∼30k papers at zero marginal cost); maturing entity resolution from exact-name blocking through alias expansion to LLM adjudication so SuperCard clusters are clean; auto-propagating a SuperCard's validated URL/email back to member cards to raise their tier; and feeding the SuperCard layer into a knowledge graph where resources are canonical nodes and papers are edges. The architecture is built so each of these consumes the *same* entity-resolution and consensus layer built once — enrichment and SuperCards are mutually reinforcing, not parallel efforts.

Source material: U.S. DOE OSTI public records (osti.gov).
All facts traceable; nothing fabricated.